



Pashov Audit Group

Paxos Labs Security Review

August 16th 2026 - August 19th 2026

Executive Summary

A time-boxed security review of Paxos Labs was conducted through the Pashov Audit Group bug-hunt program. This report documents 1 issue across the reviewed codebase, including findings reported by InfiniteSec, IvanFitro, JeRRy, K42, Schnilch, Slayer Security.

Project	Paxos Labs
Timeline	August 16th 2026 - August 19th 2026

About

Pashov Audit Group is a security research collective focused on smart-contract and protocol security. This report documents the issues surfaced during a time-boxed review delivered through our competitive bug-hunt program.

METHODOLOGY

Researchers independently reviewed the in-scope code and reported the issues they found. Each submission was then triaged and validated by the judging team; duplicates were merged and severities were finalized. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

DISCLAIMER

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue was reviewed after the hunt; the outcome is recorded in the finding's Resolution section.

Risk Classification

Critical

Loss or compromise of critical assets, or protocol-wide impact. Must be fixed before launch.

High

Significant material loss of assets, or serious harm to a group of users.

Medium

Moderate material loss, or moderate harm, often under conditional assumptions.

Low

Minor impact, limited scope, or requires unlikely conditions to trigger.

Info

No direct security impact - best-practice, code-quality or hardening suggestion.

Scope

Repo: <https://github.com/paxoslabs/pxl-boring-vault/pull/339>

Commit: 1d6b609ada6a13db6aa1aa2b55482b9e44cb2fec

Findings

SEVERITY	COUNT
Low	1
Total	1

ID	TITLE	SEVERITY	STATUS
L-01	Deleted fill history causes cross-order collateral underfunding	Low	Fixed

Low Severity

[L-01] Deleted fill history causes cross-order collateral underfunding

Low**Fixed**

SUMMARY

`CowSwapHelper.cancelOrder` treats CoW Protocol's `filledAmount(orderUid)` as permanent lifetime accounting, but the canonical settlement contract deliberately zeroes that value after an order expires via `freeFilledAmountStorage`, which any authenticated solver can trigger through a settlement self-interaction.

`placeOrder` records the original `sellAmount` and transfers that amount into the helper, and it intentionally lets every concurrent order for the same token draw from one shared helper balance through an unlimited relayer allowance:

```
orders[orderUid] = OrderRecord({
  status: OrderStatus.Active, sellToken: address(params.sellToken), sellAmount:
params.sellAmount
});
params.sellToken.safeTransferFrom(boringVault, address(this), params.sellAmount);
```

Cancellation keeps no local filled amount or isolated per-order residual. It subtracts the current external getter value from the original amount and transfers the result from the shared balance:

```
unfilled = record.sellAmount - settlement.filledAmount(orderUid);
if (unfilled != 0) {
    ERC20(record.sellToken).safeTransfer(boringVault, unfilled);
}
```

The settlement cleanup is explicitly designed to run after expiry and writes zero to the fill mapping:

```
require(validTo < block.timestamp, "GPv2: order still valid");
orderStorage[orderUid] = 0;
```

Once `filledAmount[A]` is zeroed, `cancelOrder(A)` computes `sellAmount - 0` (for example `1 ether - 0`) instead of the true remainder (`0.6 ether` after a `0.4 ether` fill), so it tries to reclaim more than that order ever left behind.

IMPACT

A protocol-supported cleanup selected by an authenticated solver corrupts the helper's per-order refund accounting. The BoringVault and the settlement contract themselves are not compromised, and tokens are never sent to the solver, but the affected orders cannot complete through their intended paths.

Two outcomes:

- Single-order case: the real remainder (for example `0.6 ether` of a `1 ether` order) stays in the helper, but ordinary cancellation reverts because it attempts to transfer the full `1 ether`, rolling back closure and signature revocation. The remainder becomes unreachable through the normal cancellation path.
- Concurrent-order case: cancelling A succeeds by consuming collateral belonging to a still-live same-token order B. B remains active and pre-signed but backed by only `0.6 ether` against its `1 ether` sell amount, so its full settlement or cancellation cannot execute.

The effect applies to any supported sell token and scales with the historical fill amount erased for the expired order. Recovery requires privileged sweeping, recapitalization, or an upgrade.

EXPLOIT PATH

1. The vault places orders A and B for the same sell token, depositing `1 ether` each into the shared helper balance.
2. A is partially filled for `0.4 ether`, leaving `0.6 ether` attributable to A and `1 ether` to B.
3. After A expires, a solver includes the normal `freeFilledAmountStorage([A])` self-interaction during settlement, zeroing `filledAmount[A]`.
4. The vault calls `cancelOrder(A)`; the helper computes `1 ether - 0` instead of `0.6 ether`.
5. Either the transfer reverts (single-order), or A's refund pulls `0.4 ether` of B's collateral, leaving B underfunded.

PREREQUISITES

The storage cleanup requires an authenticated CoW solver, but it is a normal, protocol-supported settlement interaction available on any expired order, not a privileged action against the vault. It only applies after an order has expired.

RECOMMENDATION

Do not rely on `GPv2Settlement.filledAmount` as durable post-expiry accounting. Either isolate sell-token custody per UID, or enforce one live order per sell token and reconcile refunds from an authoritative per-order residual balance tracked by the helper. Add a post-expiry finalization or recovery path that stays correct after CoW deletes its fill storage.

RESOLUTION

PAXOS LABS

Fixed. We have removed per-order accounting in favor of allowing the strategist to withdraw tokens from the helper freely. Withdrawals are sent only to the vault, so this cannot lead to loss of funds.

fixed-PAG